

Experiment: Prompt Tuning and Parameter-Efficient Fine-Tuning (PEFT) using LoRA

1. Aim

To implement **Prompt Tuning** and **Parameter-Efficient Fine-Tuning (PEFT)** using **LoRA (Low-Rank Adaptation)** on a pre-trained Transformer model and analyze the reduction in trainable parameters while maintaining effective model performance.

Requirements

Hardware Requirements

- Computer/Laptop
- Minimum 8 GB RAM
- Internet connection
- GPU recommended but not mandatory

Software Requirements

- Python 3.9 or above
- Jupyter Notebook / Google Colab
- Transformers
- Datasets
- PEFT
- Accelerate
- PyTorch

Installation

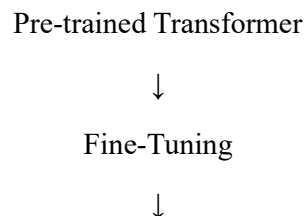
```
!pip install -q transformers datasets peft accelerate evaluate
```

Theory

Fine-Tuning

Fine-tuning means adapting a pre-trained model to a specific task by training its parameters using a task-specific dataset.

For example:



Sentiment Classification

In conventional fine-tuning, **almost all model parameters are updated**.

This can require considerable:

- GPU memory
- Training time
- Storage
- Computational resources

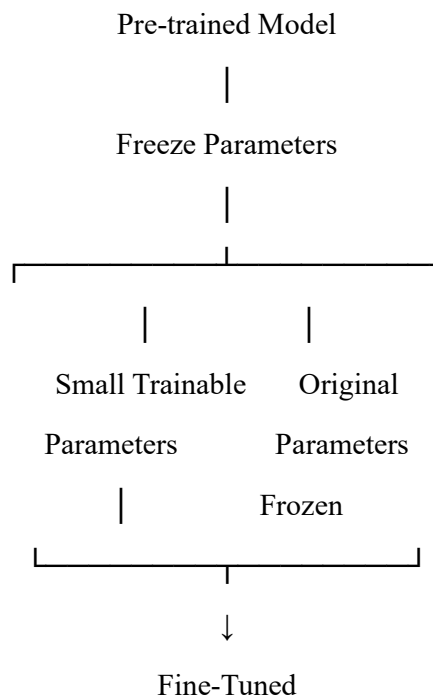
Parameter-Efficient Fine-Tuning

Parameter-Efficient Fine-Tuning (PEFT) is a technique for adapting large pre-trained models by training only a **small subset of parameters** while keeping most of the original model frozen.

Common PEFT techniques include:

1. Prompt Tuning
2. Prefix Tuning
3. Adapter Tuning
4. LoRA
5. IA³

Basic concept



Model

Prompt Tuning

Prompt Tuning adds **learnable virtual tokens** to the input.

Unlike manually designed prompts, these virtual tokens are optimized during training.

Example

Normal input:

This movie is excellent.

Prompt-tuned input:

[P1] [P2] [P3] [P4] This movie is excellent.

P1, P2, P3, P4 are trainable virtual tokens.

The original model parameters remain frozen.

Advantages

- Very few trainable parameters
- Lower memory requirement
- Suitable for large language models
- Easy to store task-specific prompt parameters

LoRA – Low-Rank Adaptation

LoRA is a popular PEFT technique.

Suppose the original model contains a weight matrix:

$$W$$

Instead of directly modifying W , LoRA represents the update as:

$$W' = W + \Delta W$$

where:

$$\Delta W = BA$$

Therefore:

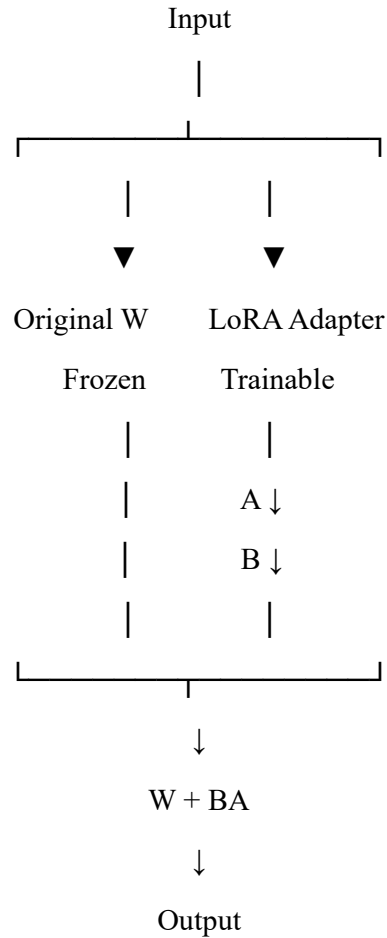
$$W' = W + BA$$

Here:

- W = original frozen weight matrix
- A = trainable low-rank matrix

- B = trainable low-rank matrix
- BA = learned weight update

LoRA Architecture



The main advantage is that **W is not updated**.

Algorithm

Algorithm: PEFT using LoRA

Input: Pre-trained Transformer model and sentiment dataset

Output: LoRA fine-tuned model

1. Start.
2. Install and import required libraries.
3. Load the IMDB sentiment dataset.
4. Load a pre-trained DistilBERT model.
5. Load the corresponding tokenizer.

6. Tokenize the input sentences.
7. Define LoRA configuration.
8. Freeze the original model parameters.
9. Add LoRA adapters to selected Transformer layers.
10. Display total and trainable parameters.
11. Train the PEFT model.
12. Evaluate the model on test data.
13. Record accuracy and loss.
14. Compare trainable parameters with full fine-tuning.
15. Stop.

Procedure

Step 1: Open Google Colab

Create a new Python notebook.

Step 2: Install libraries

```
!pip install -q transformers datasets peft accelerate evaluate
```

Step 3: Import libraries

```
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer
)
from peft import LoraConfig, get_peft_model
import numpy as np
```

Step 4: Load dataset

```
dataset = load_dataset("imdb")
print(dataset)
```

For a quick laboratory demonstration, use a smaller subset:

```
train_dataset = dataset["train"].shuffle(seed=42).select(range(2000))
test_dataset = dataset["test"].shuffle(seed=42).select(range(500))
```

Load Pre-trained Model

```
model_name = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2
)
```

Tokenization

```
def tokenize_function(examples):
```

```
    return tokenizer(
        examples["text"],
        padding="max_length",
        truncation=True,
        max_length=128
    )
```

```
train_dataset = train_dataset.map(
    tokenize_function,
    batched=True
)
```

```
test_dataset = test_dataset.map(
    tokenize_function,
    batched=True
)
```

```
train_dataset = train_dataset.remove_columns(["text"])
```

```
test_dataset = test_dataset.remove_columns(["text"])
```

```
train_dataset.set_format("torch")
```

```
test_dataset.set_format("torch")
```

Configure LoRA

```
lora_config = LoraConfig(  
    r=8,  
    lora_alpha=16,  
    lora_dropout=0.1,  
    target_modules=["q_lin", "v_lin"],  
    bias="none",  
    task_type="SEQ_CLS"  
)
```

Meaning of parameters

Parameter	Meaning
r=8	Rank of LoRA matrices
lora_alpha=16	Scaling factor
lora_dropout=0.1	Dropout applied to LoRA
target_modules	Layers where LoRA is applied
task_type	Sequence classification

Apply LoRA

```
peft_model = get_peft_model(  
    model,  
    lora_config  
)  
peft_model.print_trainable_parameters()
```

Typical output will be similar to:

trainable params: 739,586

all params: 67,694,594

trainable%: approximately 1.09%

Note: Exact values can vary with library/model versions.

Define Evaluation Metric

```
def compute_metrics(eval_pred):
    predictions, labels = eval_pred

    predictions = np.argmax(predictions, axis=1)

    accuracy = np.mean(predictions == labels)

    return {
        "accuracy": accuracy
    }
```

Define Training Arguments

```
training_args = TrainingArguments(
    output_dir="./lora_results",
    learning_rate=2e-4,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy="epoch",
    save_strategy="epoch",
    logging_steps=100,
    report_to="none"
)
```

If your installed Transformers version does not recognize eval_strategy, use evaluation_strategy="epoch" instead.

Create Trainer

```
trainer = Trainer(
    model=peft_model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
```


)

Train the LoRA Model

```
trainer.train()
```

The model updates only the LoRA parameters instead of updating the entire pre-trained model.

Evaluate the Model

```
results = trainer.evaluate()
```

```
print(results)
```

Example:

```
eval_loss: 0.32
```

```
eval_accuracy: 0.87
```

Actual results may vary depending on dataset size, hardware, random seed, and training configuration.

Save the PEFT Model

```
peft_model.save_pretrained("./lora_sentiment_model")
```

```
tokenizer.save_pretrained("./lora_sentiment_model")
```

The saved PEFT model contains the small adapter parameters rather than a completely new copy of the original model.

Optional: Compare Full Fine-Tuning and LoRA

To understand the advantage of PEFT, calculate the number of trainable parameters.

Full Fine-Tuning

```
full_model = AutoModelForSequenceClassification.from_pretrained(
```

```
    model_name,
```

```
    num_labels=2
```

```
)
```

```
total_params = sum(
```

```
    p.numel() for p in full_model.parameters()
```

```
)
```

```
trainable_params = sum(
```

```
    p.numel()
```

```
    for p in full_model.parameters()
```

```
        if p.requires_grad
    )
    print("Total Parameters:", total_params)
    print("Trainable Parameters:", trainable_params)
```

LoRA

```
peft_model.print_trainable_parameters()
```

The important observation is:

Full Fine-Tuning: Nearly 100% parameters are trainable

LoRA:

Only a small percentage of parameters are trainable

Sample Output

Dataset

Dataset:

Train samples : 2000

Test samples : 500

LoRA Parameters

trainable params: 739,586

all params: 67,694,594

trainable%: approximately 1.09%

Training

Epoch 1: Training Loss: 0.48

Epoch 2: Training Loss: 0.29

Evaluation

Evaluation Loss: 0.32

Evaluation Accuracy: 0.87

Note: These are representative sample values, not guaranteed outputs.

Observation Table

S.No.	Method	Total Parameters	Trainable Parameters	Training Cost	Accuracy
1	Full Fine-Tuning	~67.7 M	~67.7 M	High	—
2	LoRA	~67.7 M	~0.74 M	Low	~87%
3	Prompt Tuning	Model dependent	Very low	Very Low	—

Important Observation

LoRA can reduce the number of parameters that need to be updated from **millions of parameters to a small fraction of the model**, while still providing effective task adaptation.

Result: Thus, Prompt Tuning and Parameter-Efficient Fine-Tuning were studied, and LoRA was successfully implemented on a pre-trained Transformer model. The experiment demonstrated that LoRA can adapt a pre-trained model by training only a small number of additional parameters while keeping the majority of the original model parameters frozen, thereby reducing computational and memory requirements.

Experiment: Implementation of Soft Prompting using Prompt Tuning with Hugging Face PEFT

Aim: To implement **Soft Prompting using Prompt Tuning** with the **Hugging Face PEFT library** and adapt a pre-trained Transformer model for a sentiment-classification task by training only a small number of virtual prompt parameters.

Requirements

Hardware

- Computer/Laptop
- Minimum 8 GB RAM
- Internet connection
- GPU recommended
- Google Colab can be used

Software

- Python 3.9+
- Google Colab / Jupyter Notebook
- PyTorch
- Hugging Face Transformers
- Hugging Face Datasets
- Hugging Face PEFT
- Accelerate

Installation

```
!pip install -q transformers datasets peft accelerate evaluate
```

Theory

Soft Prompting

Soft Prompting is a parameter-efficient technique in which a small set of **continuous, trainable vectors** is added to the input of a pre-trained language model.

Unlike a normal prompt, soft prompts are not necessarily meaningful words.

For example, a hard prompt is:

Classify the sentiment:

This movie was excellent.

A soft prompt can be represented conceptually as:

[P1] [P2] [P3] [P4] This movie was excellent.

Here:

- P1, P2, P3, P4 are virtual prompt tokens.
- Their embeddings are learned during training.
- The original model parameters remain frozen.

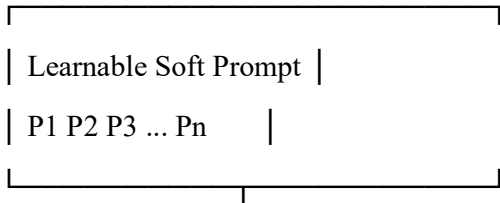
Prompt Tuning

Prompt Tuning is a PEFT technique that learns a small number of virtual prompt embeddings while keeping the pre-trained model frozen.

The process is:

Input Text

|



|



Transformer

(Frozen)

|



Prediction

|



Loss Function

|



Update Soft Prompt Only

Mathematical Representation

Let:

$$X = [x_1, x_2, \dots, x_n]$$

represent the input embeddings and:

$$P = [p_1, p_2, \dots, p_m]$$

represent the trainable soft prompt.

The model receives:

$$[P; X]$$

The Transformer parameters θ remain frozen.

Only P is optimized:

$$P^* = \arg \min_P L(f_\theta([P; X]), Y)$$

Therefore, only a very small number of parameters need to be trained.

Difference Between Hard and Soft Prompting

Feature	Hard Prompting	Soft Prompting
Prompt	Human-written words	Learned vectors
Parameters trained	Usually none	Soft prompt parameters
Human readable	Yes	No
Base model	May be frozen	Frozen
Optimization	Manual design	Gradient-based
PEFT technique	No	Yes
Computational cost	Very low	Low

Algorithm

Algorithm: Soft Prompting using Prompt Tuning

Input: Pre-trained Transformer model and sentiment dataset

Output: Prompt-tuned model

1. Start.

2. Install required Python libraries.
3. Load the sentiment dataset.
4. Select a pre-trained Transformer model.
5. Load the corresponding tokenizer.
6. Tokenize the input text.
7. Create a Prompt Tuning configuration.
8. Specify the number of virtual prompt tokens.
9. Freeze the original model parameters.
10. Add trainable virtual prompt embeddings.
11. Train the model.
12. Update only the soft prompt parameters.
13. Evaluate the model on test data.
14. Display trainable and total parameters.
15. Compare the results.
16. Stop.

Dataset

For this experiment, the **IMDB Movie Review Dataset** is used.

It contains:

- Positive movie reviews
- Negative movie reviews

The task is:

Input → Movie Review

Output → Positive / Negative

For a laboratory demonstration, we use a smaller subset to reduce training time.

Procedure

Step 1: Install Libraries

```
!pip install -q transformers datasets peft accelerate evaluate
```

Step 2: Import Libraries

```
from datasets import load_dataset
```

```
from transformers import (  
    AutoTokenizer,  
    AutoModelForSequenceClassification,  
    TrainingArguments,  
    Trainer  
)
```

```
from peft import PromptTuningConfig  
from peft import PromptTuningInit  
from peft import get_peft_model  
import numpy as np
```

Load Dataset

```
dataset = load_dataset("imdb")  
print(dataset)
```

For a quick experiment:

```
train_dataset = dataset["train"].shuffle(seed=42).select(range(2000))  
test_dataset = dataset["test"].shuffle(seed=42).select(range(500))
```

Load Pre-trained Model

For prompt tuning, a model such as **DistilBERT** can be used for demonstrating the PEFT concept.

```
model_name = "distilbert-base-uncased"  
tokenizer = AutoTokenizer.from_pretrained(model_name)  
model = AutoModelForSequenceClassification.from_pretrained(  
    model_name,  
    num_labels=2  
)
```

Tokenize Dataset

```
def tokenize_function(examples):  
    return tokenizer(  

```



```

        examples["text"],
        padding="max_length",
        truncation=True,
        max_length=128
    )
train_dataset = train_dataset.map(
    tokenize_function,
    batched=True
)
test_dataset = test_dataset.map(
    tokenize_function,
    batched=True
)

```

Remove the original text column:

```

train_dataset = train_dataset.remove_columns(["text"])
test_dataset = test_dataset.remove_columns(["text"])
train_dataset.set_format("torch")
test_dataset.set_format("torch")

```

Create Prompt Tuning Configuration

```

prompt_config = PromptTuningConfig(
    task_type="SEQ_CLS",
    prompt_tuning_init=PromptTuningInit.RANDOM,
    num_virtual_tokens=8,
    tokenizer_name_or_path=model_name
)

```

Important Parameters

Parameter	Meaning
task_type	Defines the task

Parameter	Meaning
<code>prompt_tuning_init</code>	Method used to initialize prompt embeddings
<code>num_virtual_tokens</code>	Number of learnable prompt tokens
<code>tokenizer_name_or_path</code>	Tokenizer associated with the model

Apply Prompt Tuning

```
peft_model = get_peft_model(
    model,
    prompt_config
)
```

Display the trainable parameters:

```
peft_model.print_trainable_parameters()
```

A typical result will be similar to:

```
trainable params: 6,144
```

```
all params: 67,694,594
```

```
trainable%: 0.009%
```

Note: The exact number can vary depending on the model architecture and PEFT/Transformers versions.

Define Evaluation Metric

```
def compute_metrics(eval_pred):
    predictions, labels = eval_pred

    predictions = np.argmax(
        predictions,
        axis=1
    )

    accuracy = np.mean(
        predictions == labels
    )

    return {
```

```
    "accuracy": accuracy
}
```

Define Training Parameters

```
training_args = TrainingArguments(
    output_dir="./prompt_tuning_results",
    learning_rate=3e-3,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy="epoch",
    save_strategy="epoch",
    logging_steps=100,
    report_to="none"
)
```

If an older Transformers version is installed, replace:

```
eval_strategy="epoch"
```

with:

```
evaluation_strategy="epoch"
```

Create Trainer

```
trainer = Trainer(
    model=peft_model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)
```

Train the Soft Prompt

```
trainer.train()
```

During training:

Original Transformer parameters

↓

Frozen

↓

Soft Prompt Parameters

↓

Updated

The important point is that the experiment does **not** update the entire Transformer model.

Evaluate the Model

```
results = trainer.evaluate()
```

```
print(results)
```

Possible output:

```
eval_loss: 0.45
```

```
eval_accuracy: 0.82
```

The actual result will depend on the dataset subset, model version, hyperparameters, random seed, and hardware.

Save the Soft Prompt

```
peft_model.save_pretrained(
```

```
    "./soft_prompt_model"
```

```
)
```

```
tokenizer.save_pretrained(
```

```
    "./soft_prompt_model"
```

```
)
```

The PEFT checkpoint is much smaller than saving a separately fine-tuned full model.

Complete Python Program

The following is a compact version suitable for a **lab record/notebook**.

```
!pip install -q transformers datasets peft accelerate evaluate
```

```
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer
)
```

```
from peft import (
    PromptTuningConfig,
    PromptTuningInit,
    get_peft_model
)
```

```
import numpy as np
```

```
# 1. Load Dataset
```

```
dataset = load_dataset("imdb")
```

```
train_dataset = (
    dataset["train"]
    .shuffle(seed=42)
    .select(range(2000))
)
```

```
test_dataset = (
    dataset["test"]
    .shuffle(seed=42)
    .select(range(500))
)
```

```
# 2. Load Model and Tokenizer
```

```
model_name = "distilbert-base-uncased"
```

```
tokenizer = AutoTokenizer.from_pretrained(
    model_name
)

model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2
)
```

3. Tokenization

```
def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        padding="max_length",
        truncation=True,
        max_length=128
    )

train_dataset = train_dataset.map(
    tokenize_function,
    batched=True
)

test_dataset = test_dataset.map(
    tokenize_function,
    batched=True
)

train_dataset = train_dataset.remove_columns(["text"])
test_dataset = test_dataset.remove_columns(["text"])
train_dataset.set_format("torch")
test_dataset.set_format("torch")

# 4. Prompt Tuning Configuration

prompt_config = PromptTuningConfig(
```

```
task_type="SEQ_CLS",
prompt_tuning_init=PromptTuningInit.RANDOM,
num_virtual_tokens=8,
tokenizer_name_or_path=model_name
)
```

5. Apply Prompt Tuning

```
peft_model = get_peft_model(
    model,
    prompt_config
)
peft_model.print_trainable_parameters()
```

6. Evaluation Function

```
def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(
        predictions,
        axis=1
    )
    accuracy = np.mean(
        predictions == labels
    )
    return {
        "accuracy": accuracy
    }
```

7. Training Arguments

```
training_args = TrainingArguments(
    output_dir="./prompt_tuning_results",
    learning_rate=3e-3,
    per_device_train_batch_size=8,
```

```

        per_device_eval_batch_size=8,
        num_train_epochs=2,
        weight_decay=0.01,
        eval_strategy="epoch",
        save_strategy="epoch",
        logging_steps=100,
        report_to="none"
    )

# 8. Trainer
trainer = Trainer(
    model=peft_model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)

# 9. Training
trainer.train()

# 10. Evaluation
results = trainer.evaluate()
print("\nEvaluation Results:")
print(results)

# 11. Save Model
peft_model.save_pretrained(
    "./soft_prompt_model"
)

tokenizer.save_pretrained(
    "./soft_prompt_model"
)

```


Sample Output

Dataset

Train samples: 2000

Test samples : 500

Trainable Parameters

trainable params: approximately 6,144

all params: approximately 67.7 million

trainable%: approximately 0.009%

Training

Epoch 1

Training Loss: 0.58

Epoch 2

Training Loss: 0.43

Evaluation

Evaluation Loss: 0.45

Evaluation Accuracy: 0.82

Note: Sample values are illustrative. Students should record the actual values obtained when executing the program.

Observation Table

S.No.	Parameter	Observation
1	Dataset	IMDB
2	Task	Sentiment Classification
3	Pre-trained Model	DistilBERT
4	Number of Virtual Tokens	8
5	Total Parameters	~67.7 M
6	Trainable Parameters	Very small fraction
7	Base Model Parameters	Frozen
8	Learning Rate	3×10^{-3}

S.No. Parameter		Observation
9	Epochs	2
10	Test Accuracy	Record actual value

Result:

Soft Prompting using Prompt Tuning was successfully implemented, and the model was evaluated for sentiment classification with significantly fewer trainable parameters than conventional fine-tuning.